

2026 CCF 非专业级别软件能力认证第一轮
(CSP-S1) 提高级 C++语言试题

认证时间：2026 年 9 月 26 日 14:30~16:30

考生注意事项：

- 试题纸共有 20 页，答题纸共有 1 页，满分 100 分。请在答题纸上作答，写在试题纸上的一律无效。
- 不得使用任何电子设备（如计算器、手机、电子词典等）或查阅任何书籍资料。

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 下列 Linux 命令中，用于连接并显示文件内容的是（ ）
A. cp
B. cat
C. ping
D. touch
2. 已知某用二叉链表存储（即每个结点包含数据域、左孩子指针和右孩子指针）的哈夫曼树共有 m 个叶子结点，则该哈夫曼树中总共有多少个空指针域？（ ）
A. m
B. $m+1$
C. $2m$
D. $2m+1$
3. 对于一棵包含 n 个结点的完全二叉树，进行如下操作：对每个叶子结点，从该结点出发向上访问直到根结点，统计路径上经过的结点数之和。假设每个结点的访问时间为 $O(1)$ ，则完成上述操作的总时间复杂度为（ ）
A. $O(n)$
B. $O(n \log n)$
C. $O(n^2)$
D. $O(n^2 \log n)$
4. 小 h 从原点 $(0,0)$ 出发，每一步可以向右 $(1,0)$ 或向上 $(0,1)$ 走，需要到达点 $(5,5)$ 。要求行走过程中始终满足 $x \geq y$ （即路径上的任何一点，向右走的步数不少于向上走的步数）。满足条件的路径有多少条？（ ）
A. 252
B. 90
C. 132
D. 42
5. 用两个栈模拟一个队列，入队时元素统一压入栈 A。以下关于出队操作的说法，正确的是（ ）

- A. 出队时，若栈 B 为空，则将栈 A 中全部元素转移到栈 B，再从栈 B 弹出栈顶；若栈 B 非空，则直接从栈 B 弹出栈顶
- B. 出队时，无论栈 B 是否为空，都先将栈 A 中全部元素转移到栈 B，再从栈 B 弹出栈顶
- C. 出队时，直接在栈 A 中删除最早入队的那个元素（即栈 A 的栈底元素）
- D. 出队时，直接从栈 A 弹出栈顶元素
6. 某人上楼梯，每一步可以跨 1 级或 2 级台阶。要登上 10 级台阶，共有（ ）种不同的走法。
- A. 55
- B. 89
- C. 144
- D. 233
7. 在 C++ 中，关于 `std::set<int>` 与 `std::map<int, int>` 的说法，正确的是（ ）
- A. `set` 中的元素可以重复
- B. `map` 按照元素插入的顺序存储键值对
- C. `set` 中的元素按从小到大的顺序自动有序，插入、删除、查找的时间复杂度均为 $O(\log n)$
- D. `map` 的键（key）可以重复
8. 关于有向无环图的拓扑排序，下列说法正确的是（ ）
- A. 任何有向无环图的拓扑排序都是唯一的
- B. 含有环的有向图也可以进行拓扑排序
- C. 若图中存在一条有向边 $u \rightarrow v$ ，则任一拓扑排序中 u 一定排在 v 之前
- D. 拓扑排序的结果就是该图的深度优先遍历序列
9. 在使用 KMP 算法进行字符串匹配时，`next` 数组（部分匹配表）的作用是（ ）
- A. 记录模式串中每个位置匹配失败后应回溯到的位置
- B. 记录主串中每个字符出现的次数
- C. 记录主串的后缀数组
- D. 在匹配失败时跳过主串中所有不可能匹配的位置，与模式串无关
10. 对一个 n 个顶点、 m 条边的带权有向图用 Dijkstra 算法计算单源最短路时，如果不使用堆或其它优先队列进行优化，则其时间复杂度为（ ）
- A. $O(n^2)$
- B. $O(m \log n)$
- C. $O((m+n) \log n)$
- D. $O(mn)$
11. 求以下表达式的值（已知 20260913 是质数）：
 $(1/2 + 1/3 + 1/4 + \dots + 1/20260912)^{20260913} \bmod 20260913$
结果是（ ）
- A. 0
- B. 1

C. $2^{20260913} \bmod 20260913$

D. 20260912

12. 在一条长度为 2 的线段上随机取两个点，则以这两个点为端点的线段的期望长度是 ()

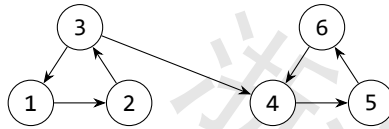
A. 1

B. $1/3$

C. $2/3$

D. $3/4$

13. 下图中含有节点数大于 1 的强连通分量的个数是 ()



A. 3

B. 6

C. 4

D. 2

14. 1 到 200 之间，不能被 2、3、7 中任何一个整除的整数共有 () 个

A. 57

B. 58

C. 59

D. 60

15. 根结点深度为 1，则一棵深度为 h 的满 k 叉树 ($k > 1$) 的结点数为 ()

A. $(k^h - 1)/(k - 1)$

B. $(k^{(h+1)} - 1)/(k - 1)$

C. $k^h/(k - 1)$

D. k^h

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填 $\checkmark$ ，错误填 $\times$ ；除特殊说明外，判断题 1.5 分，选择题 3 分，共计 40 分）

(1)

```
01#include <bits/stdc++.h>
02using namespace std;
03const int M = 2e6 + 5;
04long long A[200005], B[200005], pA[200005], pB
    [200005], F[M], G[M];
05int n, cx, cy;
06long long D;
07long long C(long long *a, long long *p, long long x)
    {
08    int k = lower_bound(a, a + n, x) - a;
09    long long L = k > 0 ? p[k - 1] : 0, s = 0;
10    if (k > 0)
11        s += (long long)k * x - L;
12    if (k < n)
13        s += p[n - 1] - L - (long long)(n - k) * x;
14    return s;
15}
16int main() {
17    ios::sync_with_stdio(false);
18    cin.tie(0);
19    cin >> n >> D;
20    for (int i = 0; i < n; i++)
21        cin >> A[i] >> B[i];
22    sort(A, A + n);
23    sort(B, B + n);
24    pA[0] = A[0];
25    pB[0] = B[0];
26    for (int i = 1; i < n; i++)
27        pA[i] = pA[i - 1] + A[i], pB[i] = pB[i - 1]
+ B[i];
28    for (long long x = A[n - 1] - D; x <= A[0] + D;
x++) {
```

```

29         long long v = C(A, pA, x);
30         if (v <= D)
31             F[cx++] = v;
32     }
33     for (long long y = B[n - 1] - D; y <= B[0] + D;
y++) {
34         long long v = C(B, pB, y);
35         if (v <= D)
36             G[cy++] = v;
37     }
38     sort(F, F + cx);
39     sort(G, G + cy);
40     long long ans = 0;
41     for (int i = 0, j = cy - 1; i < cx; i++) {
42         while (j >= 0 && F[i] + G[j] > D)
43             j--;
44         if (j < 0)
45             break;
46         ans += j + 1;
47     }
48     cout << ans;
49 }

```

● 判断题

16. 函数 C 的功能是计算所有 $|A[i]-x|$ 的和, 即数组 A 中所有元素与 x 的距离之和。()
A. 正确
B. 错误
17. 若将枚举 x 的循环改为从 $A[0]-D$ 到 $A[n-1]+D$ (其余部分不变), 程序的输出一定发生改变。()
A. 正确
B. 错误
18. 当 $x < A[0]-D$ 时, 不可能有满足 $\sum(|A[i]-x|) \leq D$ 的 x, 因此枚举 x 从 $A[n-1]-D$ 开始是充分的。()
A. 正确
B. 错误

● 单选题

19. 函数 C 中 $k = \text{lower_bound}(a, a+n, x) - a$ 表示 ()

- A. a 中小于 x 的元素个数
- B. a 中不大于 x 的元素个数
- C. a 中等于 x 的元素个数
- D. a 中大于 x 的元素个数

20. 当输入为

2 3

0 0

1 0

时, 程序的输出为 ()

- A. 6
- B. 7
- C. 8
- D. 10

(2)

```
01#include <bits/stdc++.h>
02using namespace std;
03const int N = 6e2 + 5;
04int g[N][N], w[N], v[N], id[N], n, m;
05int main() {
06    ios::sync_with_stdio(0);
07    cin.tie(0);
08    cin >> n >> m;
09    for (int i = 1; i <= n; ++i)
10        id[i] = i;
11    for (int i = 0, x, y, z; i < m; ++i) {
12        cin >> x >> y >> z;
13        g[x][y] += z;
14        g[y][x] += z;
15    }
16    int ans = n < 2 ? 0 : 2e9, cur = n;
17    while (cur > 1) {
18        memset(w, 0, sizeof(w));
19        memset(v, 0, sizeof(v));
20        int p = 0, u = 0;
21        for (int i = 1; i <= cur; ++i) {
22            u = -1;
23            for (int j = 1; j <= cur; ++j)
24                if (!v[id[j]] && (u == -1 || w[id[j]
25                ]] > w[u]))
26                    u = id[j];
27            v[u] = 1;
28            if (i == cur) {
29                if (w[u] < ans)
30                    ans = w[u];
31                for (int j = 1; j <= cur; ++j) {
32                    int t = id[j];
33                    if (t != u)
34                        g[p][t] += g[u][t], g[t][p]
```

```

    += g[t][u];
34         }
35         for (int j = 1; j <= cur; ++j)
36             if (id[j] == u) {
37                 id[j] = id[cur];
38                 break;
39             }
40         --cur;
41     } else {
42         p = u;
43         for (int j = 1; j <= cur; ++j)
44             if (!v[id[j]])
45                 w[id[j]] += g[u][id[j]];
46     }
47 }
48 }
49 cout << ans;
50 }

```

● 判断题

21. 该程序求出的是无向连通图的全局最小割（删去后使图不再连通的最小边权和）。()

- A. 正确
- B. 错误

22. 当输入的 $n=1$ 时，程序的输出为 0。()

- A. 正确
- B. 错误

23. 主循环 `while (cur > 1)` 恰好执行 n 轮。()

- A. 正确
- B. 错误

● 单选题

24. 在每一轮循环中，最后被加入集合的顶点 u 对应的 $w[u]$ 表示 ()

- A. u 与所有未加入集合的顶点之间的边权和
- B. u 与已加入集合的顶点之间的边权和（即当前划分的割的大小）
- C. u 的度数
- D. u 与上一个加入集合的顶点之间的边权和

25. 该算法的时间复杂度为 ()

- A. $O(n^2)$

B. $O(n^3)$

C. $O(nm)$

D. $O(m \log n)$

26. 当输入为

4 6

1 2 5

1 3 1

2 4 1

3 4 2

2 3 1

1 4 2

时，程序的输出为（ ）

A. 3

B. 4

C. 5

D. 6

(3)

```
01#include <bits/stdc++.h>
02using namespace std;
03
04const int N = 8e5 + 5;
05
06struct Q {
07    int t, op, x, y, id;
08    long long v;
09} q[N], p[N];
10
11int n, m, tot, qc, a[N];
12long long c[N], ans[N];
13set<int> S[N];
14
15void add(int x, long long v) {
16    for (; x; x -= x & -x)
17        c[x] += v;
18}
19
20long long ask(int x) {
21    long long s = 0;
22    for (; x < N; x += x & -x)
23        s += c[x];
24    return s;
25}
26
27void add_e(int u, int v, int w) {
28    if (u && v)
29        q[++tot] = {tot, 0, v, u, 0, 1LL * w * (v -
30    u)};
31
32void cdq(int l, int r) {
33    if (l >= r)
```

```

34         return;
35     int m = (l + r) / 2;
36     cdq(l, m);
37     cdq(m + 1, r);
38     int j = l;
39     for (int i = m + 1; i <= r; ++i) {
40         if (q[i].op) {
41             while (j <= m && q[j].x <= q[i].x) {
42                 if (!q[j].op)
43                     add(q[j].y, q[j].v);
44                 j++;
45             }
46             ans[q[i].id] += ask(q[i].y);
47         }
48     }
49     for (int k = l; k < j; ++k)
50         if (!q[k].op)
51             add(q[k].y, -q[k].v);
52     int i = l, k = l;
53     j = m + 1;
54     while (i <= m && j <= r)
55         p[k++] = (q[i].x <= q[j].x ? q[i++] : q[j
56         ++]);
57     while (i <= m)
58         p[k++] = q[i++];
59     while (j <= r)
60         p[k++] = q[j++];
61     for (i = l; i <= r; ++i)
62         q[i] = p[i];
63 }
64 int main() {
65     cin.tie(0)->sync_with_stdio(0);
66     cin >> n >> m;
67     for (int i = 1; i <= n; ++i) {

```

```

68     cin >> a[i];
69     S[a[i]].insert(i);
70 }
71 for (int i = 1; i <= n; ++i) {
72     int pr = 0;
73     for (int x : S[i]) {
74         add_e(pr, x, 1);
75         pr = x;
76     }
77 }
78 while (m--) {
79     int op, x, y;
80     cin >> op >> x >> y;
81     if (op == 1) {
82         if (a[x] == y)
83             continue;
84         int c1 = a[x], c2 = y;
85         auto it = S[c1].find(x);
86         int pr = (it == S[c1].begin() ? 0 : *
prev(it));
87         auto nit = next(it);
88         int nx = (nit == S[c1].end() ? 0 : *nit)
;
89         add_e(pr, x, -1);
90         add_e(x, nx, -1);
91         add_e(pr, nx, 1);
92         S[c1].erase(it);
93
94         S[c2].insert(x);
95         it = S[c2].find(x);
96         pr = (it == S[c2].begin() ? 0 : *prev(it
));
97         nit = next(it);
98         nx = (nit == S[c2].end() ? 0 : *nit);
99         add_e(pr, nx, -1);

```

```

100         add_e(pr, x, 1);
101         add_e(x, nx, 1);
102         a[x] = y;
103     } else {
104         q[++tot] = {tot, 1, y, x, ++qc, 0};
105     }
106 }
107 cdq(1, tot);
108 for (int i = 1; i <= qc; ++i)
109     cout << ans[i] << "\n";
110 }

```

● 判断题

27. 该程序采用分治的方法，离线处理所有的修改与查询操作。()
 A. 正确
 B. 错误
28. 函数 ask(x) 查询的是树状数组中所有下标不大于 x 的位置的元素之和。()
 A. 正确
 B. 错误
29. 若查询区间 [1, r] 内某种形状恰好出现一次,则该形状对该区间记忆值的贡献为 0。()
 A. 正确
 B. 错误

● 单选题

30. 函数 add_e(u, v, w) 中 if (u && v) 的作用是 ()
 A. 仅当 u 与 v 均非 0 时,才生成一个事件
 B. 当 u 与 v 中至少一个为 0 时,生成一个事件
 C. 当 u 与 v 相等时,生成一个事件
 D. 当 u 与 v 均非 0 时,不生成事件
31. 该程序分治部分的时间复杂度约为 ()
 A. $O(n+m)$
 B. $O((n+m) \log(n+m))$
 C. $O((n+m) \log^2(n+m))$
 D. $O((n+m)^2)$
32. 当输入为
 7 6
 1 2 3 1 3 2 1
 2 3 7
 2 1 3

1 7 2

1 3 2

2 1 6

2 5 7

时，第一个查询的输出为（ ）

A. 3

B. 5

C. 7

D. 9

三、完善程序（单选题，每小题 3 分，共计 30 分）

(1)（矩阵变换）给出一个 N 行 M 列的矩阵 A ，满足：① $M > N$ ；②矩阵中每个数都是 $[0, N]$ 中的自然数；③每行中 $[1, N]$ 中每个自然数恰好出现一次（因此每行中 0 恰好出现 $M-N$ 次）；④每列中 $[1, N]$ 中每个自然数至多出现一次。现在要在每行中选取一个非零数，并把这个数之后的数都赋值为这个数，希望在操作后性质④仍然成立。若存在方案，输出每行选取的数（应为 1 到 N 的一个排列）；否则输出任意卖萌表情。上述参数满足 $N \leq 200, M < 400, T < 50$ 。

提示：该问题可转化为稳定婚姻问题——每行与每个数字作为两类参与者，行 i 按数字 x 在该行中出现的位置 $p[i][x]$ 从小到大排序，数字 v 按其在各行中位置从大到小排序；再用队列依次尝试匹配，发生冲突时按双方偏好决定替换谁。

```
01#include <bits/stdc++.h>
02using namespace std;
03const int E = 4e2 + 5, Q = 4e4 + 5;
04int n, m, a[E][E], p[E][E], pr[E][E], pn[E][E], rk[E
    ][E], rn[E][E], mt[E],
05    mn[E], q[Q], ptr[E];
06int main() {
07    ios::sync_with_stdio(0);
08    cin.tie(0);
09    int T;
10    cin >> T;
11    while (T--) {
12        cin >> n >> m;
13        for (int i = 1; i <= n; ++i)
14            for (int j = 1; j <= m; ++j) {
15                cin >> a[i][j];
16                if (a[i][j])
17                    p[i][a[i][j]] = j;
18            }
19        for (int i = 1; i <= n; ++i) {
20            iota(pr[i] + 1, pr[i] + n + 1, 1);
21            sort(pr[i] + 1, pr[i] + n + 1,
22                [&](int x, int y) { return
                ①; });
23            pr[i][n + 1] = 0;
24        }
```

```

25     for (int v = 1; v <= n; ++v) {
26         iota(pn[v] + 1, pn[v] + n + 1, 1);
27         sort(pn[v] + 1, pn[v] + n + 1,
28             [&](int i, int j) { return
②      ; });
29         for (int k = 1; k <= n; ++k)
30             ③      ;
31     }
32     fill(mt + 1, mt + n + 1, 0);
33     fill(mn + 1, mn + n + 1, 0);
34     fill(ptr + 1, ptr + n + 1, 1);
35     int h = 0, t = 0, ok = 1;
36     for (int i = 1; i <= n; ++i)
37         q[t++] = i;
38     while (h < t) {
39         int i = q[h++], v = pr[i][ptr[i]++];
40         if (!v) {
41             ok = 0;
42             break;
43         }
44         if (!mn[v]) {
45             mn[v] = i;
46             mt[i] = v;
47         } else {
48             int j = mn[v];
49             if (④      ) {
50                 mn[v] = i;
51                 mt[i] = v;
52                 mt[j] = 0;
53                 ⑤      ;
54             } else
55                 q[t++] = i;
56         }
57     }
58     if (!ok)

```

```

59         cout << "(^o^)/\n";
60     else
61         for (int i = 1; i <= n; ++i)
62             cout << mt[i] << " \n"[i == n];
63     }
64 }

```

33. ①处应填 ()
- A. return p[i][x] < p[i][y]; B. return p[i][x] > p[i][y];
 C. return p[i][x] == p[i][y]; D. return p[i][x] - p[i][y];
34. ②处应填 ()
- A. return p[i][v] < p[j][v]; B. return p[i][v] > p[j][v];
 C. return p[i][v] == p[j][v]; D. return p[i][v] - p[j][v];
35. ③处应填 ()
- A. rn[v][pn[v][k]] = k; B. rn[v][k] = pn[v][k];
 C. rn[pn[v][k]][v] = k; D. rn[v][pn[v][k]] = pn[v][k];
36. ④处应填 ()
- A. if (rn[v][i] < rn[v][j]) B. if (rn[v][i] > rn[v][j])
 C. if (rn[v][i] == rn[v][j]) D. if (rn[i][v] < rn[j][v])
37. ⑤处应填 ()
- A. q[t++] = i; B. q[t++] = j;
 C. q[h++] = j; D. q[t] = j;

(2) (树上咒语相撞) 给定一棵 n 个结点的树 (结点编号 1 到 n)，其中有 $n-1$ 条无向道路。问有多少个三元组 (u, v, w) 满足: u, v, w 互不相同, 且存在从 u 到 w 与从 v 到 w 的两条路径, 这两条路径不经过相同的道路 (即 w 是 u 到 v 路径上的中间点, 且 u, v 位于 w 的不同分支)。同时, 会不断在树中加入新边 (新边可能连接已存在的结点对, 也可能连接同一个结点自身), 每加入一条新边后, 需要输出当前满足条件的三元组个数。上述参数满足 $n, q \leq 10^5$ 。

提示: 三元组计数可按 w 分类: u, v 必须落在 w 的不同的相邻分支中。加边后, 环上的点会被并查集合并成一个分量, 维护每个分量的大小 z 与”锚点子树”大小 u , 并利用原树信息 s 更新答案。

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 typedef long long ll;
04 const int E = 1e5 + 5;
05 int n, hd[E], to[E * 2], ne[E * 2], cnt, fa[E], dep[
    E], sz[E], f[E];
06 ll z[E], s[E], u[E], ans;
07 void add(int x, int y) { to[++cnt] = y, ne[cnt] = hd
    [x], hd[x] = cnt; }
08 void dfs(int x, int p) {
09     fa[x] = p, dep[x] = dep[p] + 1, sz[x] = 1;
10     for (int i = hd[x]; i; i = ne[i])
11         if (to[i] != p)
12             dfs(to[i], x), ①;
13 }
14 int find(int x) { return f[x] == x ? x : f[x] = find
    (f[x]); }
15 void merge(int x, int y) {
16     x = find(x), y = find(y);
17     if (x == y)
18         return;
19     ll cx = z[x], cy = z[y], tx = u[x], ty = n - tx;
20     ans -= cx * (cx - 1) * (cx - 2) + 2 * cx * (cx -
        1) * (n - cx) + cx * ((n - cx) * (n - cx) - s[x
        ]);
21     ans -= cy * (cy - 1) * (cy - 2) + 2 * cy * (cy -
```

```

    1) * (n - cy) + cy * ((n - cy) * (n - cy) - s[y
]);
22  s[y] = s[y] - tx * tx + s[x] - ty * ty;
23  ⑤;
24  f[x] = y;
25  cx = z[y];
26  ans += cx * (cx - 1) * (cx - 2) + 2 * cx * (cx -
    1) * (n - cx) + cx * ((n - cx) * (n - cx) - s[y
]);
27}
28int main() {
29  ios::sync_with_stdio(0);
30  cin.tie(0);
31  cin >> n;
32  for (int i = 1; i < n; ++i) {
33      int x, y;
34      cin >> x >> y;
35      add(x, y);
36      add(y, x);
37  }
38  dfs(1, 0);
39  for (int i = 1; i <= n; ++i)
40      f[i] = i, z[i] = 1, ③;
41  for (int i = 1; i <= n; ++i) {
42      ll t = 0;
43      for (int j = hd[i]; j; j = ne[j])
44          if (to[j] != fa[i])
45              t += ②;
46      if (fa[i])
47          t += 1ll * (n - sz[i]) * (n - sz[i]);
48      s[i] = t;
49  }
50  for (int i = 1; i <= n; ++i) {
51      ll c = z[i];
52      ans += c * (c - 1) * (c - 2) + 2 * c * (c -

```

```

1) * (n - c) + c * ((n - c) * (n - c) - s[i]);
53     }
54     cout << ans << '\n';
55     int q;
56     cin >> q;
57     while (q--) {
58         int x, y;
59         cin >> x >> y;
60         x = find(x), y = find(y);
61         while (x != y) {
62             if (____④____)
63                 swap(x, y);
64             int p = find(fa[x]);
65             merge(x, p);
66             x = find(x);
67         }
68         cout << ans << '\n';
69     }
70 }

```

38. ①处应填 ()

- A. sz[x] += sz[x]; B. sz[x] += sz[to[i]];
 C. sz[to[i]] += sz[x]; D. sz[x] = sz[to[i]];

39. ②处应填 ()

- A. t += sz[to[j]] * sz[to[j]]; B. t += 111 * sz[to[j]] * sz[to[j]];
 C. t += 111 * sz[to[j]] * sz[to[i]]; D. t += 111 * sz[i] * sz[to[j]];

40. ③处应填 ()

- A. u[i] = i; B. u[i] = sz[i];
 C. u[i] = dep[i]; D. u[i] = n;

41. ④处应填 ()

- A. if (dep[x] < dep[y]) swap(x, y); B. if (dep[x] > dep[y]) swap(x, y);
 C. if (sz[x] < sz[y]) swap(x, y); D. if (fa[x] < fa[y]) swap(x, y);

42. ⑤处应填 ()

- A. z[y] += z[x]; B. z[x] += z[y];
 C. z[y] = z[x]; D. f[y] = x;